

LC 2392 Build Matrix with Conditions

Google, Meta

1. Problem Understanding & Rules

Given numbers from **1 to k**

Row conditions: Eg: $1 \rightarrow 2$ means **1 must appear above 2**.

$3 \rightarrow 2$ means **3 must appear above 2**.

Similar for col condition: Eg: $2 \rightarrow 1$ means **2 must be left of 1**.

Note: both the vertical and horizontal order are completely indep

2. Logic & Core Approach (Hinglish)

```
// basic aadya ye he ki  
// ek baar kahn row order pe laga lunga  
// ek baar kahn col order pe laga lunga  
// phir dono ko merge kr dunha  
// iske liye har ek number ka rownumber aur column number kya tha  
// ye pata rkhna padega
```

3. Visual Dry Run

(S1) Kahn on Row graph

let that return [3 1 2 4]

(we haven't yet decided column order)

3
1
2

(S2) Similarly let Kahn on col graph return

[2 4 1 3]

2	4	1	3
---	---	---	---

4

Overall → (K x K matrix)

0	0	0	3
0	0	1	0
2	0	0	0
0	4	0	0

```
// ab maan lo roworder = 3 1 2 4 // value of row
//           0 1 2 3 // index of row

// hame pata hona chahiye har value kis row aur kis column me he
```

4. Implementation

```
class Solution {
public:

    vector<int> kahn(vector<vector<int>>& adj, vector<int>& indeg, int k) {
        queue<int> q;
        vector<int> ans;
        for(int i = 1; i <= k; i++) {
            if(indeg[i] == 0) {
                q.push(i);
            }
        }

        while(!q.empty()) {
            int curr = q.front();
            q.pop();
            ans.push_back(curr);

            for(auto &it : adj[curr]) {
                indeg[it]--;
                if(indeg[it] == 0) {
                    q.push(it);
                }
            }
        }
    }
};
```

```
}
```

```
    return ans;
```

```
}
```

```
vector<vector<int>> buildMatrix(int k, vector<vector<int>>& rowConditions,  
vector<vector<int>>& colConditions) {
```

```
    // basic aadya ye he ki
```

```
    // ek baar kahn row order pe laga lunga
```

```
    // ek baar kahn col order pe laga lunga
```

```
    // phir dono ko merge kr dunha
```

```
    // iske liye har ek number ka rownumber aur column number kya tha
```

```
    // ye pata rkhna padega
```

```
    vector<vector<int>> radj(k + 1);
```

```
    vector<int> rindeg(k + 1, 0);
```

```
    vector<vector<int>> cadj(k + 1);
```

```
    vector<int> cindeg(k + 1, 0);
```

```
    for(auto &it : rowConditions) {
```

```
        int u = it[0];
```

```
        int v = it[1];
```

```
        radj[u].push_back(v);
```

```
        rindeg[v]++;
```

```
    }
```

```
    for(auto &it : colConditions) {
```

```
        int u = it[0];
```

```
        int v = it[1];
```

```
        cadj[u].push_back(v);
```

```
        cindeg[v]++;
```

```
    }
```

```
    vector<int> roworder = kahn(radj, rindeg, k);
```

```
    if(roworder.size() != k) return {};
```

```
    vector<int> colorder = kahn(cadj, cindeg, k);
```

```
    if(colorder.size() != k) return {};
```

```
    // ab maan lo roworder = 3 1 2 4 // value of row
```

```
    //                          0 1 2 3 // index of row
```

```
    // hame pata hona chahiye har value kis row aur kis column me he
```

```
vector<int> rowpos(k + 1);
vector<int> colpos(k + 1);

for(int i = 0; i < k; i++) {
    rowpos[roworder[i]] = i;
    colpos[colorder[i]] = i;
}

vector<vector<int>> ans(k, vector<int>(k, 0)); // k*k ki matrix for
returning ans

// (S3)
for(int i = 1; i <= k; i++) {
    ans[rowpos[i]][colpos[i]] = i;
}

return ans;
}
};
```